

Setting Up

Generated by Doxygen 1.9.4

1 Data Structure Index	1
1.1 Data Structures	1
2 File Index	3
2.1 File List	3
3 Data Structure Documentation	5
3.1 map_s Struct Reference	5
3.1.1 Detailed Description	5
3.1.2 Field Documentation	5
3.1.2.1 bigger_pos	5
3.1.2.2 bigger_size	6
3.1.2.3 height	6
3.1.2.4 map	6
3.1.2.5 width	6
4 File Documentation	7
4.1 include/my.h File Reference	7
4.1.1 Function Documentation	8
4.1.1.1 my_compute_power_rec()	8
4.1.1.2 my_compute_square_root()	9
4.1.1.3 my_find_prime_sup()	9
4.1.1.4 my_getnbr()	9
4.1.1.5 my_is_prime()	9
4.1.1.6 my_isneg()	9
4.1.1.7 my_put_error()	9
4.1.1.8 my_put_nbr()	10
4.1.1.9 my_putchar()	10
4.1.1.10 my_putstr()	10
4.1.1.11 my_revstr()	10
4.1.1.12 my_showmem()	10
4.1.1.13 my_showstr()	11
4.1.1.14 my_sort_int_array()	11
4.1.1.15 my_str_isalpha()	11
4.1.1.16 my_str_islower()	11
4.1.1.17 my_str_isnum()	11
4.1.1.18 my_str_isprintable()	12
4.1.1.19 my_str_isupper()	12
4.1.1.20 my_str_to_word_array()	12
4.1.1.21 my_strcapitalize()	12
4.1.1.22 my_strcat()	12
4.1.1.23 my_strcmp()	13
4.1.1.24 my_strcpy()	13

4.1.1.25 my_strdup()	13
4.1.1.26 my_strlen()	13
4.1.1.27 my_strlowcase()	13
4.1.1.28 my_strncat()	14
4.1.1.29 my_strncmp()	14
4.1.1.30 my_strncpy()	14
4.1.1.31 my_strstr()	14
4.1.1.32 my_strupcase()	14
4.1.1.33 my_swap()	15
4.2 my.h	15
4.3 include/setting_up.h File Reference	15
4.3.1 Typedef Documentation	16
4.3.1.1 map_t	16
4.3.2 Function Documentation	16
4.3.2.1 set_map_map()	16
4.3.2.2 set_map_pattern()	17
4.3.2.3 setting_up()	18
4.4 setting_up.h	18
4.5 src/main.c File Reference	19
4.5.1 Function Documentation	19
4.5.1.1 error_handling()	19
4.5.1.2 main()	20
4.5.1.3 main_aux()	20
4.6 src/set_map.c File Reference	21
4.6.1 Function Documentation	21
4.6.1.1 set_map_map()	21
4.6.1.2 set_map_pattern()	22
4.7 src/setting_up.c File Reference	22
4.7.1 Function Documentation	22
4.7.1.1 setting_up()	22

Index	25
--------------	-----------

Chapter 1

Data Structure Index

1.1 Data Structures

Here are the data structures with brief descriptions:

map_s	Structure représentant la map et ses propriétés	5
-----------------------	---	-------------------

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

include/my.h	7
include/setting_up.h	15
src/main.c	19
src/set_map.c	21
src/setting_up.c	22

Chapter 3

Data Structure Documentation

3.1 map_s Struct Reference

Structure représentant la map et ses propriétés.

```
#include <setting_up.h>
```

Data Fields

- char * [map](#)
- size_t [height](#)
- size_t [width](#)
- size_t [bigger_pos](#)
- size_t [bigger_size](#)

3.1.1 Detailed Description

Structure représentant la map et ses propriétés.

3.1.2 Field Documentation

3.1.2.1 bigger_pos

```
size_t map_s::bigger_pos
```

Position du plus grand carré

3.1.2.2 bigger_size

```
size_t map_s::bigger_size
```

Taille du plus grand carré

3.1.2.3 height

```
size_t map_s::height
```

Hauteur de la map

3.1.2.4 map

```
char* map_s::map
```

Tableau représentant la map

3.1.2.5 width

```
size_t map_s::width
```

Largeur de la map

The documentation for this struct was generated from the following file:

- [include/setting_up.h](#)

Chapter 4

File Documentation

4.1 include/my.h File Reference

This graph shows which files directly or indirectly include this file:

Functions

- int [my_compute_power_rec](#) (int nb, int p)
Élève nb à la puissance p (récursif).
- int [my_compute_square_root](#) (int nb)
Calcule la racine carrée entière de nb.
- int [my_find_prime_sup](#) (int nb)
Retourne le plus petit nombre premier $\geq$ nb.
- int [my_getnbr](#) (char const *str)
Convertit une chaîne en entier.
- int [my_isneg](#) (int n)
Affiche 'N' si n est négatif, 'P' sinon.
- int [my_is_prime](#) (int nb)
Vérifie si nb est un nombre premier.
- int [my_putchar](#) (char c)
Affiche un caractère sur la sortie standard.
- int [my_put_nbr](#) (int nb)
Affiche un nombre sur la sortie standard.
- int [my_putstr](#) (char const *str)
Affiche une chaîne sur la sortie standard.
- char * [my_revstr](#) (char *str)
Inverse une chaîne de caractères.
- int [my_showmem](#) (char const *str, int size)
Affiche une chaîne en hexadécimal (mémoire).
- int [my_showstr](#) (char const *str)
Affiche une chaîne avec caractères non imprimables en hexadécimal.
- void [my_sort_int_array](#) (int *array, int size)
Trie un tableau d'entiers.
- char * [my_strcapitalize](#) (char *str)
Met la première lettre de chaque mot en majuscule.

- char * [my_strcat](#) (char *dest, char const *src)
Concatène src à la fin de dest.
- int [my_strcmp](#) (char const *s1, char const *s2)
Compare deux chaînes de caractères.
- char * [my_strcpy](#) (char *dest, char const *src)
Copie src dans dest.
- int [my_str_isalpha](#) (char const *str)
Vérifie si la chaîne ne contient que des lettres.
- int [my_str_islower](#) (char const *str)
Vérifie si la chaîne ne contient que des minuscules.
- int [my_str_isnum](#) (char const *str)
Vérifie si la chaîne ne contient que des chiffres.
- int [my_str_isprintable](#) (char const *str)
Vérifie si la chaîne ne contient que des caractères imprimables.
- int [my_str_isupper](#) (char const *str)
Vérifie si la chaîne ne contient que des majuscules.
- int [my_strlen](#) (char const *str)
Retourne la longueur d'une chaîne.
- char * [my_strlowercase](#) (char *str)
Met tous les caractères d'une chaîne en minuscules.
- char * [my_strncat](#) (char *dest, char const *src, int nb)
Concatène src à dest sur nb caractères max.
- int [my_strncmp](#) (char const *s1, char const *s2, int n)
Compare deux chaînes sur n caractères.
- char * [my_strncpy](#) (char *dest, char const *src, int n)
Copie n caractères de src dans dest.
- char * [my_strstr](#) (char *str, char const *to_find)
Cherche to_find dans str.
- char * [my_strupcase](#) (char *str)
Met tous les caractères d'une chaîne en majuscules.
- void [my_swap](#) (int *a, int *b)
Échange deux entiers.
- char * [my_strdup](#) (char const *src)
Duplique une chaîne de caractères.
- char ** [my_str_to_word_array](#) (char const *str)
Découpe une chaîne en mots (tableau).
- int [my_put_error](#) (const char *err_msg)
Affiche un message d'erreur sur la sortie d'erreur.

4.1.1 Function Documentation

4.1.1.1 my_compute_power_rec()

```
int my_compute_power_rec (
    int nb,
    int p )
```

Élève nb à la puissance p (récursif).

4.1.1.2 my_compute_square_root()

```
int my_compute_square_root (
    int nb )
```

Calcule la racine carrée entière de nb.

4.1.1.3 my_find_prime_sup()

```
int my_find_prime_sup (
    int nb )
```

Retourne le plus petit nombre premier $\geq$ nb.

4.1.1.4 my_getnbr()

```
int my_getnbr (
    char const * str )
```

Convertit une chaîne en entier.

4.1.1.5 my_is_prime()

```
int my_is_prime (
    int nb )
```

Vérifie si nb est un nombre premier.

4.1.1.6 my_isneg()

```
int my_isneg (
    int n )
```

Affiche 'N' si n est négatif, 'P' sinon.

4.1.1.7 my_put_error()

```
int my_put_error (
    const char * err_msg )
```

Affiche un message d'erreur sur la sortie d'erreur.

4.1.1.8 my_put_nbr()

```
int my_put_nbr (
    int nb )
```

Affiche un nombre sur la sortie standard.

4.1.1.9 my_putchar()

```
int my_putchar (
    char c )
```

Affiche un caractère sur la sortie standard.

4.1.1.10 my_putstr()

```
int my_putstr (
    char const * str )
```

Affiche une chaîne sur la sortie standard.

4.1.1.11 my_revstr()

```
char * my_revstr (
    char * str )
```

Inverse une chaîne de caractères.

4.1.1.12 my_showmem()

```
int my_showmem (
    char const * str,
    int size )
```

Affiche une chaîne en hexadécimal (mémoire).

4.1.1.13 my_showstr()

```
int my_showstr (
    char const * str )
```

Affiche une chaîne avec caractères non imprimables en hexadécimal.

4.1.1.14 my_sort_int_array()

```
void my_sort_int_array (
    int * array,
    int size )
```

Trie un tableau d'entiers.

4.1.1.15 my_str_isalpha()

```
int my_str_isalpha (
    char const * str )
```

Vérifie si la chaîne ne contient que des lettres.

4.1.1.16 my_str_islower()

```
int my_str_islower (
    char const * str )
```

Vérifie si la chaîne ne contient que des minuscules.

4.1.1.17 my_str_isnum()

```
int my_str_isnum (
    char const * str )
```

Vérifie si la chaîne ne contient que des chiffres.

4.1.1.18 my_str_isprintable()

```
int my_str_isprintable (
    char const * str )
```

Vérifie si la chaîne ne contient que des caractères imprimables.

4.1.1.19 my_str_isupper()

```
int my_str_isupper (
    char const * str )
```

Vérifie si la chaîne ne contient que des majuscules.

4.1.1.20 my_str_to_word_array()

```
char ** my_str_to_word_array (
    char const * str )
```

Découpe une chaîne en mots (tableau).

4.1.1.21 my_strcapitalize()

```
char * my_strcapitalize (
    char * str )
```

Met la première lettre de chaque mot en majuscule.

4.1.1.22 my_strcat()

```
char * my_strcat (
    char * dest,
    char const * src )
```

Concatène src à la fin de dest.

4.1.1.23 my_strcmp()

```
int my_strcmp (
    char const * s1,
    char const * s2 )
```

Compare deux chaînes de caractères.

4.1.1.24 my_strcpy()

```
char * my_strcpy (
    char * dest,
    char const * src )
```

Copie src dans dest.

4.1.1.25 my_strdup()

```
char * my_strdup (
    char const * src )
```

Duplique une chaîne de caractères.

4.1.1.26 my_strlen()

```
int my_strlen (
    char const * str )
```

Retourne la longueur d'une chaîne.

4.1.1.27 my_strlowercase()

```
char * my_strlowercase (
    char * str )
```

Met tous les caractères d'une chaîne en minuscules.

4.1.1.28 my_strncat()

```
char * my_strncat (
    char * dest,
    char const * src,
    int nb )
```

Concatène src à dest sur nb caractères max.

4.1.1.29 my_strncmp()

```
int my_strncmp (
    char const * s1,
    char const * s2,
    int n )
```

Compare deux chaînes sur n caractères.

4.1.1.30 my_strncpy()

```
char * my_strncpy (
    char * dest,
    char const * src,
    int n )
```

Copie n caractères de src dans dest.

4.1.1.31 my_strstr()

```
char * my_strstr (
    char * str,
    char const * to_find )
```

Cherche to_find dans str.

4.1.1.32 my_strupcase()

```
char * my_strupcase (
    char * str )
```

Met tous les caractères d'une chaîne en majuscules.

4.1.1.33 my_swap()

```
void my_swap (
    int * a,
    int * b )
```

Échange deux entiers.

4.2 my.h

[Go to the documentation of this file.](#)

```
1 /*
2 ** EPITECH PROJECT, 2024
3 ** my.h
4 ** File description:
5 ** my.h
6 */
7
8 #ifndef MY_H_
9 #define MY_H_
10
11 int my_compute_power_rec(int nb, int p);
12 int my_compute_square_root(int nb);
13 int my_find_prime_sup(int nb);
14 int my_getnbr(char const *str);
15 int my_isneg(int n);
16 int my_is_prime(int nb);
17 int my_putchar(char c);
18 int my_put_nbr(int nb);
19 int my_putstr(char const *str);
20 char *my_revstr(char *str);
21 int my_showmem(char const *str, int size);
22 int my_showstr(char const *str);
23 void my_sort_int_array(int *array, int size);
24 char *my_strcapitalize(char *str);
25 char *my_strcat(char *dest, char const *src);
26 int my_strcmp(char const *s1, char const *s2);
27 char *my_strcpy(char *dest, char const *src);
28 int my_str_isalpha(char const *str);
29 int my_str_islower(char const *str);
30 int my_str_isnum(char const *str);
31 int my_str_isprintable(char const *str);
32 int my_str_isupper(char const *str);
33 int my_strlen(char const *str);
34 char *my_strlowercase(char *str);
35 char *my_strncat(char *dest, char const *src, int nb);
36 int my_strncmp(char const *s1, char const *s2, int n);
37 char *my_strncpy(char *dest, char const *src, int n);
38 char *my_strstr(char *str, char const *to_find);
39 char *my_strupcase(char *str);
40 void my_swap(int *a, int *b);
41 char *my_strdup(char const *src);
42 char **my_str_to_word_array(char const *str);
43 int my_put_error(const char *err_msg);
44 #endif /* MY_H_ */
```

4.3 include/setting_up.h File Reference

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <limits.h>
#include "my.h"
```

Include dependency graph for setting_up.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [map_s](#)

Structure représentant la map et ses propriétés.

Typedefs

- typedef struct [map_s](#) [map_t](#)

Structure représentant la map et ses propriétés.

Functions

- int [setting_up](#) ([map_t](#) *map)
Détermine le plus grand carré de '.' dans la map et le marque.
- int [set_map_map](#) ([map_t](#) *map, char **argv, char *buffer, struct stat *filestat)
Initialise la map à partir d'un fichier.
- int [set_map_pattern](#) ([map_t](#) *map, char *size, char *pattern)
Crée une map carrée en répétant un motif.

4.3.1 Typedef Documentation

4.3.1.1 map_t

```
typedef struct map\_s map\_t
```

Structure représentant la map et ses propriétés.

4.3.2 Function Documentation

4.3.2.1 set_map_map()

```
int set_map_map (
    map\_t * map,
    char ** argv,
    char * buffer,
    struct stat * filestat )
```

Initialise la map à partir d'un fichier.

Parameters

<i>map</i>	Structure map à remplir
<i>argv</i>	Arguments du programme
<i>buffer</i>	Buffer source
<i>filestat</i>	Statistiques du fichier

Returns

0 en cas de succès, code d'erreur sinon

Initialise la map à partir d'un fichier.

Parameters

<i>map</i>	Structure map à remplir
<i>argv</i>	Arguments du programme
<i>buffer</i>	Buffer à remplir
<i>filestat</i>	Statistiques du fichier

Returns

0 en cas de succès, 84 sinon

4.3.2.2 set_map_pattern()

```
int set_map_pattern (  
    map_t * map,  
    char * size,  
    char * pattern )
```

Crée une map carrée en répétant un motif.

Parameters

<i>map</i>	Structure map à remplir
<i>size</i>	Taille de la map
<i>pattern</i>	Motif à utiliser

Returns

0 en cas de succès, code d'erreur sinon

Crée une map carrée en répétant un motif.

Parameters

<i>map</i>	Structure map à remplir
<i>size</i>	Taille de la carte (hauteur et largeur identiques)
<i>pattern</i>	Motif à répliquer dans la carte

Returns

0 en cas de succès, 84 sinon

4.3.2.3 setting_up()

```
int setting_up (
    map_t * map )
```

Détermine le plus grand carré de '.' dans la map et le marque.

Parameters

<i>map</i>	Structure map à traiter
------------	-------------------------

Returns

0 en cas de succès, code d'erreur sinon

Détermine le plus grand carré de '.' dans la map et le marque.

Parameters

<i>map</i>	Structure map
------------	---------------

Returns

0

4.4 setting_up.h

[Go to the documentation of this file.](#)

```
1 /*
2 ** EPITECH PROJECT, 2024
3 ** setting_up.h
4 ** File description:
5 ** setting_up.h
6 */
7
8 #ifndef SETTING_UP_H_
9 #define SETTING_UP_H_
10
11 #include <stdio.h>
12 #include <unistd.h>
13 #include <stdlib.h>
14 #include <sys/types.h>
15 #include <sys/stat.h>
16 #include <fcntl.h>
17 #include <limits.h>
18 #include "my.h"
19
20 typedef struct map_s {
21     char *map;
22     size_t height;
23     size_t width;
24     size_t bigger_pos;
25     size_t bigger_size;
26 } map_t;
27
28 int setting_up(map_t *map);
29
30 int set_map_map(map_t *map,
```

```

48     char **argv, char *buffer, struct stat *filestat);
49
57 int set_map_pattern(map_t *map, char *size, char *pattern);
58
59 #endif /* SETTING_UP_H_ */

```

4.5 src/main.c File Reference

#include "setting_up.h"
 Include dependency graph for main.c:

Functions

- int [error_handling](#) (int argc, char **argv, char **buffer, struct stat *filestat)
Valide les arguments et réserve le buffer adapté selon que l'entrée est un fichier ou un motif à générer.
- int [main_aux](#) (map_t *map, int argc, char **argv, char **buffer)
Initialise la structure map en fonction des arguments reçus.
- int [main](#) (int argc, char **argv)
Point d'entrée principal du programme.

4.5.1 Function Documentation

4.5.1.1 error_handling()

```

int error_handling (
    int argc,
    char ** argv,
    char ** buffer,
    struct stat * filestat )

```

Valide les arguments et réserve le buffer adapté selon que l'entrée est un fichier ou un motif à générer.

Parameters

<i>argc</i>	Nombre d'arguments
<i>argv</i>	Tableau d'arguments
<i>buffer</i>	Pointeur vers le buffer à allouer
<i>filestat</i>	Informations sur le fichier à lire

Returns

1 si un motif doit être généré, 0 si un fichier est utilisé, 84 en cas d'erreur

4.5.1.2 main()

```
int main (
    int argc,
    char ** argv )
```

Point d'entrée principal du programme.

Alloue la structure principale, lance l'initialisation de la map puis exécute l'algorithme de résolution.

Parameters

<i>argc</i>	Nombre d'arguments
<i>argv</i>	Tableau d'arguments

Returns

0 en cas de succès, 84 en cas d'erreur

4.5.1.3 main_aux()

```
int main_aux (
    map_t * map,
    int argc,
    char ** argv,
    char ** buffer )
```

Initialise la structure map en fonction des arguments reçus.

Selon les paramètres, la carte est lue depuis un fichier ou bien générée à partir d'un motif.

Parameters

<i>map</i>	Structure map à remplir
<i>argc</i>	Nombre d'arguments
<i>argv</i>	Tableau d'arguments
<i>buffer</i>	Pointeur vers le buffer à allouer

Returns

0 en cas de succès, 84 en cas d'erreur

4.6 src/set_map.c File Reference

```
#include "setting_up.h"
```

Include dependency graph for set_map.c:

Functions

- int [set_map_pattern](#) ([map_t](#) *map, char *size, char *pattern)
Génère une carte carrée à partir d'un motif répété.
- int [set_map_map](#) ([map_t](#) *map, char **argv, char *buffer, struct stat *filestat)
Charge une carte depuis un fichier.

4.6.1 Function Documentation

4.6.1.1 set_map_map()

```
int set_map_map (  
    map\_t * map,  
    char ** argv,  
    char * buffer,  
    struct stat * filestat )
```

Charge une carte depuis un fichier.

Initialise la map à partir d'un fichier.

Parameters

<i>map</i>	Structure map à remplir
<i>argv</i>	Arguments du programme
<i>buffer</i>	Buffer à remplir
<i>filestat</i>	Statistiques du fichier

Returns

0 en cas de succès, 84 sinon

4.6.1.2 set_map_pattern()

```
int set_map_pattern (
    map_t * map,
    char * size,
    char * pattern )
```

Génère une carte carrée à partir d'un motif répété.

Crée une map carrée en répétant un motif.

Parameters

<i>map</i>	Structure map à remplir
<i>size</i>	Taille de la carte (hauteur et largeur identiques)
<i>pattern</i>	Motif à répliquer dans la carte

Returns

0 en cas de succès, 84 sinon

4.7 src/setting_up.c File Reference

```
#include "setting_up.h"
```

Include dependency graph for setting_up.c:

Functions

- int [setting_up](#) (map_t *map)

Algorithme principal : calcule les tailles de carré possibles et place le plus grand dans la carte.

4.7.1 Function Documentation

4.7.1.1 setting_up()

```
int setting_up (
    map_t * map )
```

Algorithme principal : calcule les tailles de carré possibles et place le plus grand dans la carte.

Détermine le plus grand carré de '.' dans la map et le marque.

Parameters

<i>map</i>	Structure map
------------	---------------

Returns

0

Index

- bigger_pos
 - map_s, 5
- bigger_size
 - map_s, 5
- error_handling
 - main.c, 19
- height
 - map_s, 6
- include/my.h, 7, 15
- include/setting_up.h, 15, 18
- main
 - main.c, 19
- main.c
 - error_handling, 19
 - main, 19
 - main_aux, 20
- main_aux
 - main.c, 20
- map
 - map_s, 6
- map_s, 5
 - bigger_pos, 5
 - bigger_size, 5
 - height, 6
 - map, 6
 - width, 6
- map_t
 - setting_up.h, 16
- my.h
 - my_compute_power_rec, 8
 - my_compute_square_root, 8
 - my_find_prime_sup, 9
 - my_getnbr, 9
 - my_is_prime, 9
 - my_isneg, 9
 - my_put_error, 9
 - my_put_nbr, 9
 - my_putchar, 10
 - my_putstr, 10
 - my_revstr, 10
 - my_showmem, 10
 - my_showstr, 10
 - my_sort_int_array, 11
 - my_str_isalpha, 11
 - my_str_islower, 11
 - my_str_isnum, 11
 - my_str_isprintable, 11
 - my_str_isupper, 12
 - my_str_to_word_array, 12
 - my_strcapitalize, 12
 - my_strcat, 12
 - my_strcmp, 12
 - my_strcpy, 13
 - my_strdup, 13
 - my_strlen, 13
 - my_strlowercase, 13
 - my_strncat, 13
 - my_strncmp, 14
 - my_strncpy, 14
 - my_strstr, 14
 - my_strupcase, 14
 - my_swap, 14
- my_compute_power_rec
 - my.h, 8
- my_compute_square_root
 - my.h, 8
- my_find_prime_sup
 - my.h, 9
- my_getnbr
 - my.h, 9
- my_is_prime
 - my.h, 9
- my_isneg
 - my.h, 9
- my_put_error
 - my.h, 9
- my_put_nbr
 - my.h, 9
- my_putchar
 - my.h, 10
- my_putstr
 - my.h, 10
- my_revstr
 - my.h, 10
- my_showmem
 - my.h, 10
- my_showstr
 - my.h, 10
- my_sort_int_array
 - my.h, 11
- my_str_isalpha
 - my.h, 11
- my_str_islower
 - my.h, 11
- my_str_isnum

- my.h, [11](#)
- my_str_isprintable
 - my.h, [11](#)
- my_str_isupper
 - my.h, [12](#)
- my_str_to_word_array
 - my.h, [12](#)
- my_strcapitalize
 - my.h, [12](#)
- my_strcat
 - my.h, [12](#)
- my_strcmp
 - my.h, [12](#)
- my_strcpy
 - my.h, [13](#)
- my_strdup
 - my.h, [13](#)
- my_strlen
 - my.h, [13](#)
- my_strlowercase
 - my.h, [13](#)
- my_strncat
 - my.h, [13](#)
- my_strncmp
 - my.h, [14](#)
- my_strncpy
 - my.h, [14](#)
- my_strstr
 - my.h, [14](#)
- my_strupcase
 - my.h, [14](#)
- my_swap
 - my.h, [14](#)
- set_map.c
 - set_map_map, [21](#)
 - set_map_pattern, [21](#)
- set_map_map
 - set_map.c, [21](#)
 - setting_up.h, [16](#)
- set_map_pattern
 - set_map.c, [21](#)
 - setting_up.h, [17](#)
- setting_up
 - setting_up.c, [22](#)
 - setting_up.h, [18](#)
- setting_up.c
 - setting_up, [22](#)
- setting_up.h
 - map_t, [16](#)
 - set_map_map, [16](#)
 - set_map_pattern, [17](#)
 - setting_up, [18](#)
- src/main.c, [19](#)
- src/set_map.c, [21](#)
- src/setting_up.c, [22](#)
- width
 - map_s, [6](#)